

PM_Collegio v3 · package web

Documentazione tecnica del secondo front-end (HTML + Bootstrap) servito da un mini server HTTP embedded

Modulo : web **Progetto** : PM_Collegio v3 **Ateneo** : Università di Pavia **Autore** : Giresse Kengne

1. Introduzione

Il package `it.collegio.web` aggiunge a PM_Collegio v3 un **secondo front-end** oltre alle 15 JFrame Swing del package `views`. Si tratta di un'**interfaccia web** servita da un mini server HTTP embedded, pensata per dimostrare empiricamente i vantaggi dell'architettura MVC + DAO introdotta in v2.

L'idea di fondo è semplice: se la separazione tra View, Controller, DAO e DTO è ben fatta, deve essere possibile **scrivere un secondo front-end senza toccare il dominio applicativo**. Il package `web` è la prova pratica di questa affermazione.

Cosa non è questo package :

- Non è un framework MVC web (no Spring, no Servlet API, no Jakarta EE).
- Non sostituisce le JFrame: le due interfacce coesistono e usano lo stesso DB.
- Non è pensato per la produzione: è un prototipo didattico monoutente.

1.1 Obiettivi del package

1. **Dimostrare la riusabilità dei Controller** : mostrare che lo stesso Controller può servire sia la view Swing che la view HTML, senza modifiche.
2. **Dimostrare la riusabilità dei DAO** : nessuna query SQL nuova, tutti gli accessi al DB passano dagli stessi DAO della v2 Swing.
3. **Validare il refactor v1 → v2** : in v1 (architettura monolitica) questo lavoro avrebbe richiesto la riscrittura di tutta la logica applicativa, per ogni nuovo front-end. In v3 il front-end è sostituibile.
4. **Offrire un'esperienza moderna** : interfaccia responsive Bootstrap 5, accessibile da qualunque browser sulla stessa macchina.

1.2 Numeri del lavoro

Per dare un'idea di quanto sia stata ridotta la duplicazione del codice:

Metrica	Valore
Nuove query SQL	0
Nuove classi DAO	0
Nuove classi Controller	0
Metodi aggiunti ai Controller esistenti	1 (<code>RegistrationController.getIndirizzi()</code>)
Logiche di business duplicate	0
Transazioni multi-step riscritte	0

2. Architettura

Il flusso di una richiesta dal browser fino al DB è quello che segue. Da notare che i passaggi 4 e 5 sono *esattamente* gli stessi della view Swing.

```

[Browser]
|
| GET /rooms
v
[HttpServer (porta 8080)]
| routing su context "/rooms"
v
[RoomsHandler] ===== specifico di web
| controller.getRooms()
v
[ManageRoomController] ===== riusato 1:1 dalla v3 Swing
| roomDao.getAll()
v
[RoomDao] ===== riusato 1:1 dalla v3 Swing
| QueryContainer.queryRoomGetAll
v
[DatabaseConnection]
| JDBC -> MySQL
v
[Database PM_Collegio]
|
| ResultSet -> List<Room>
|
+----- ritorna l'oggetto fino al RoomsHandler -----+
|
v
[HtmlRenderer] mescola
List<Room> con il template
rooms.html, sostituisce
{{rows}} con i <tr> HTML
|
v
HTTP 200 + text/html
|
v
[Browser]

```

2.1 I tre livelli del package

Livello	Compito	File coinvolti
Trasporto	Aprire la porta HTTP, dispatchare le URL sui giusti handler, parseare form e cookie.	<code>WebServer</code> , <code>HandlerUtils</code> , <code>StaticHandler</code>
Sessione	Verifica che l'utente sia loggato (rileggendo <code>SessionContext</code>), emette e cancella il cookie di sessione web.	<code>WebSession</code>
Presentazione	Per ogni endpoint chiama il Controller corretto e compone l'HTML iniettando i dati nei template.	15 handler in <code>handlers/</code> , 17 template in <code>templates/</code> , <code>HtmlRenderer</code>

2.2 Cosa è stato volutamente tenuto fuori

Il package è rimasto deliberatamente minimale per non snaturare il progetto universitario. In particolare:

- **Nessun framework esterno Java** (Spring, Servlet, Jersey, ecc.). Si usa solo `com.sun.net.httpserver` , già presente nel JDK.
- **Nessun template engine** (Thymeleaf, Freemarker, JSP). C'è un mini-renderer di una decina di righe che fa solo sostituzione di placeholder.
- **Nessun database aggiuntivo** per le sessioni web: il cookie HTTP punta logicamente al solito `SessionContext` in memoria.
- **Nessun build aggiuntivo**: i file HTML e CSS vivono sotto `src/` e vengono copiati in `build/classes/` da Ant standard di NetBeans.

3. Stack tecnologico

3.1 Lato server (Java)

Componente	Versione/Origine	Uso
<code>com.sun.net.httpserver.HttpServer</code>	JDK 8+	Server HTTP embedded, una sola classe.
<code>HttpHandler</code>	JDK 8+	Interfaccia implementata da ogni handler.
<code>Executors.newFixedThreadPool(4)</code>	JDK	Pool di thread che serve le richieste.
<code>java.awt.Desktop</code>	JDK	Apri automaticamente il browser allo startup.
<code>java.net.URLDecoder</code> / <code>URLEncoder</code>	JDK	Parsing dei form e dei parametri di query string.

Niente dipendenze esterne Java. Il `build.xml` di NetBeans non è stato modificato.

3.2 Lato client (HTML/CSS/JS)

Componente	Versione	Origine
Bootstrap CSS	5.3.3	CDN jsdelivr
Bootstrap JS bundle	5.3.3	CDN jsdelivr (caricato solo dove servono modali/collapse)
CSS personalizzato	n/a	<code>/static/style.css</code>

Connessione internet richiesta : i CSS e JS di Bootstrap sono caricati via CDN. Senza connessione l'interfaccia perde stili e i modali (paga fattura, modifica prenotazione) non funzionano. Per un uso completamente offline si può scaricare Bootstrap sotto `/static/` e cambiare il `href` in `layout.html`.

4. Struttura del package

Tutti i file vivono sotto `src/it/collegio/web/`. L'organizzazione è la seguente :

```
web/
├── WebServer.java          avvia HttpServer e registra le route
├── HtmlRenderer.java       legge template + sostituisce {{placeholder}}
├── WebSession.java         cookie + verifica login
├── handlers/
│   ├── HandlerUtils.java   utility: sendHtml, redirect, parseForm
│   ├── LoginHandler.java
│   ├── RegistrationHandler.java
│   ├── PasswordResetHandler.java
│   ├── LogoutHandler.java
│   ├── HomeHandler.java
│   ├── RoomsHandler.java
│   ├── ManageUserHandler.java
│   ├── ManageTenantHandler.java
│   ├── ManageReservationHandler.java
│   ├── CheckInHandler.java
│   ├── CheckOutHandler.java
│   ├── FatturaHandler.java
│   ├── GestioneFattureHandler.java
│   ├── LogsHandler.java
│   ├── LoadingHandler.java
│   ├── Loading1Handler.java
│   └── StaticHandler.java  serve i file statici (CSS, immagini)
├── templates/              risorse classpath, lette via getResourceAsStream
│   ├── layout.html         scheletro con navbar, slot {{content}}
│   ├── login.html, registration.html, password_step1.html, password_step2.html
│   ├── home.html, rooms.html, manage_user.html, manage_tenant.html
│   ├── manage_reservation.html, checkin.html, checkout.html
│   ├── fattura.html, gestione_fatture.html, logs.html
│   └── loading.html, loading1.html
└── static/
    └── style.css            palette verde PMC, tile della home, card
```

Il file `nbproject/project.properties` contiene `build.classes.excludes=**/*.java,**/*.form`, quindi i `.html` e i `.css` vengono automaticamente copiati nella cartella delle classi compilate, dove sono raggiungibili come *classpath resources*.

5. Avvio del server

5.1 Prerequisiti

- JDK 8 o superiore (lo stesso del resto di v3).
- MySQL configurato come per la v3 Swing (vedi `DatabaseConnection.java`).
- Connessione internet per caricare Bootstrap dal CDN (oppure copia locale sotto `static/`).
- Porta TCP `8080` libera sulla macchina.

5.2 Lancio da NetBeans

1. Aprire il progetto `PM_Collegiov3` in NetBeans.
2. Nel *Projects* espandere `it.collegio.web` e fare tasto destro su `WebServer.java`.
3. Selezionare **Run File** (Shift + F6).
4. Si apre automaticamente il browser predefinito su `http://localhost:8080/`.

Per chiudere il server è sufficiente premere il quadratino rosso di *Stop* nella console di NetBeans.

5.3 Lancio da riga di comando

Dopo *Clean and Build* è sufficiente lanciare la classe direttamente:

```
java -cp build/classes;lib/mysql-connector-j.jar it.collegio.web.WebServer
```

L'output atteso sulla console è:

```
PM_Collegio v3 HTML view in ascolto su http://localhost:8080/
```

Il server **non blocca la JFrame Swing**. In teoria si possono tenere aperte entrambe le UI sullo stesso DB. Attenzione però: condividono `SessionContext` in memoria, quindi se l'utente di Swing fa logout, anche la sessione web cade.

6. Routing ed endpoint

Tutte le route registrate da `WebServer.main()` :

Route	Metodi	Handler	Auth ?
/	GET	LoadingHandler	no
/loading1	GET	Loading1Handler	no
/login	GET POST	LoginHandler	no
/registration	GET POST	RegistrationHandler	no
/password	GET POST	PasswordResetHandler	no
/logout	GET	LogoutHandler	sì
/home	GET	HomeHandler	sì
/rooms	GET POST	RoomsHandler	sì (AS, AC)
/users	GET POST	ManageUserHandler	sì
/tenants	GET POST	ManageTenantHandler	sì (AS)
/reservation	GET POST	ManageRiservationHandler	sì
/checkin	GET POST	CheckInHandler	sì
/checkout	GET POST	CheckOutHandler	sì
/fattura	GET POST	FatturaHandler	sì

Route	Metodi	Handler	Auth ?
/gestione-fatture	GET POST	GestioneFattureHandler	sì
/logs	GET	LogsHandler	sì (AS, AC)
/static/*	GET	StaticHandler	no

6.1 Convenzione GET / POST

Tutti gli handler che modificano dati seguono il **pattern Post-Redirect-Get** oppure rispondono direttamente con la pagina aggiornata più un alert di esito. Le GET sono sempre *idempotenti* (puro fetch di lettura).

6.2 Parametri di query string supportati

Alcuni endpoint accettano parametri opzionali in query string:

Endpoint	Parametro	Uso
/rooms	?edit=42	Pre-popola il form di modifica per la camera 42.
/users	?edit=XXX001MARIO	Pre-popola il form con i dati dell'utente XXX001MARIO.
/users	?searchEmail=mario@x	Cerca un utente per email ed evidenzia la riga.
/tenants	?edit=3	Pre-popola il form di modifica per il committente 3.
/checkin	? searchEmail= ... &prezzo= ...	Pre-popola il form check-in con i dati del cliente e filtra le camere.
/fattura	?id=18	Mostra il dettaglio della fattura 18.

7. Componenti core

7.1 WebServer

Punto di ingresso del package. Crea un `HttpServer` su porta `8080`, registra tutte le route, gli associa il pool di thread e prova ad aprire il browser con `Desktop.browse()`.

```
HttpServer server = HttpServer.create(new InetSocketAddress(PORT), 0);
server.createContext("/login", new LoginHandler());
// ... altre 15 route
server.setExecutor(Executors.newFixedThreadPool(4));
server.start();
```

Una scelta progettuale: il `setExecutor` usa un pool fisso di 4 thread. Questo basta per un demo locale e impedisce comportamenti imprevedibili dovuti a creazione illimitata di thread.

7.2 HtmlRenderer

È il "template engine" minimo del package. Espone tre metodi:

Metodo	Uso
<code>render(template, values)</code>	Legge un file HTML dal classpath e sostituisce ogni occorrenza di <code>{{chiave}}</code> con il valore corrispondente nella mappa.
<code>renderInLayout(content, title, contentValues, layoutValues)</code>	Renderizza prima il template di contenuto, poi lo inietta dentro <code>layout.html</code> al posto di <code>{{content}}</code> .
<code>escape(s)</code>	Escape HTML standard (<code>&</code> , <code><</code> , <code>></code> , <code>"</code> , <code>'</code>). Da usare per ogni dato letto dal DB prima di iniettarlo in HTML.

Limitazione voluta : `HtmlRenderer` non supporta cicli (`{{#each ...}}`) né condizioni (`{{#if ...}}`). Per i cicli (es. le righe di una tabella) si compone la stringa nel handler in Java e la si inietta come un unico placeholder, ad esempio `{{rows}}` . Questa scelta tiene il renderer sotto le 50 righe ed evita di importare un template engine completo.

7.3 WebSession

Si occupa di tre cose, in poche righe:

- Leggere il cookie `PM_SID` dalla richiesta.
- Emettere un cookie di sessione al primo login (UUID random).
- Cancellare il cookie al logout.

La verifica del "sono loggato ?" è delegata a `SessionContext.isLoggedIn()` della v3 Swing. Questo significa che il cookie web e la sessione applicativa *non sono associati uno a uno*: il cookie è più che altro un marker, la sessione vera vive in memoria.

7.4 HandlerUtils

Utility statiche comuni a tutti gli handler:

- `sendHtml(ex, html)` — manda la risposta con `Content-Type: text/html; charset=utf-8` .
- `redirect(ex, location)` — HTTP 302 con header `Location` .
- `notFound(ex)` — HTTP 404 testuale.
- `parseForm(ex)` — legge il body `application/x-www-form-urlencoded` e lo torna come `Map<String,String>` .

7.5 StaticHandler

Serve qualsiasi file sotto `/it/collegio/web/static/` via `getResourceAsStream` . Aggiunge il giusto `Content-Type` in base all'estensione (css, js, png, jpg, svg). Se la risorsa non esiste, 404.

8. Handler in dettaglio

Ogni handler segue lo stesso pattern minimale:

1. Verificare l'autenticazione (tranne login, registration, password, loading).
2. Se POST: parsare il form, chiamare il Controller, comporre l'alert di esito.

3. Caricare i dati da mostrare via Controller.
4. Mappare i dati nel template e rispondere HTML.

8.1 LoginHandler

GET POST /login

Controller : `LoginController.login(email, password)`

Template : `login.html` (pagina full-screen senza navbar)

Flusso : il GET mostra il form; il POST chiama `LoginController.login()`, che a sua volta valida le credenziali, scrive in `AccessLog`, crea una sessione (token UUID + scadenza 8h) e popola `SessionContext`. Su successo, `WebSession.issueCookie()` + redirect a `/home`.

8.2 RegistrationHandler

GET POST /registration

Controller : `RegistrationController.registra( ... )`, `.getIndirizzi()`

Template : `registration.html`

Note : in GET popola due select — via (da `getIndirizzi()`) e domanda di sicurezza (lista hardcoded di 5 valori, identici a `questionComboBox` di RegistrationView Swing). In POST chiama `registra()`; se ok l'account viene creato con stato `ATTESA`.

8.3 PasswordResetHandler

GET

POST

/password

Controller : `PasswordResetController.getDomandaSicurezza()` , `.resetPassword()`

Template : `password_step1.html` (chiede email), `password_step2.html` (mostra domanda + chiede risposta + nuova password)

Flusso : due step, esattamente come la PassWordView Swing. Step 1: l'utente inserisce l'email, viene mostrata la sua domanda di sicurezza. Step 2: l'utente risponde e indica la nuova password; se la risposta è corretta, la password è aggiornata via

`UserDao.updatePasswordByEmail()` .

8.4 HomeHandler

GET

/home

Template : `home.html`

Note : replica esatta di `HomeView.configureButtonsForRole()` ma in HTML. Le 8 tile (Manage Room, Manage Reservation, Manage User, Manage Tenant, Check-in, Check-out, Gestione fatture, Access Logs) sono mostrate o nascoste in base al

`SessionContext.roleType` .

8.5 RoomsHandler

GET

POST

/rooms

Controller : `ManageRoomController`

Template : `rooms.html`

Operazioni supportate :

- GET = lista camere + form per Add (id auto da `getNextId()`)
- GET `?edit=ID` = form precompilato per Update
- POST `action=add` = `addRoom()`
- POST `action=update` = `updateRoom()` (bloccato lato Controller se la camera è `OCCUPATA`)
- POST `action=delete` = `deleteRoom()` con conferma JS

Accesso : solo ruoli AS e AC.

8.6 ManageUserController

GET

POST

/users

Controller : `ManageUserController`

Template : `manage_user.html`

Operazioni : GET = lista filtrata per ruolo (per il ruolo U, solo se stesso); GET `?searchEmail= ...` = ricerca con highlight e form pre-popolato; POST con `action=add | update | delete` .

Nota : l'email del form di update è `readonly` perché identifica univocamente l'utente da modificare lato Controller.

8.7 ManageTenantHandler

GET

POST

/tenants

Controller : `ManageTenantController`

Template : `manage_tenant.html`

Operazioni : GET = lista committenti; GET `?edit=ID` = form Update; POST `action=add | update` . Le select "Gestore" e "Indirizzo" sono popolate da `getAdminEmails()` e `getIndirizzi()` .

Accesso : solo AS (come da `HomeView.configureButtonsForRole`).

8.8 ManageRiservationHandler

GET

POST

/reservation

Controller : `ReservationController`

Template : `manage_reservation.html`

Operazioni : GET = lista filtrata per ruolo (ruolo U vede solo le proprie prenotazioni); POST = `aggiornaConStorico()` , transazione UPDATE Reservation + INSERT StoricoPrenotazioni in un solo commit.

UI : riga «Modifica» cliccabile, apre un sotto-form Bootstrap collapse con le date e le note editabili. Per il ruolo U, il form di modifica è nascosto.

8.9 CheckInHandler

GET

POST

/checkin

Controller : `CheckInController`

Template : `checkin.html`

Flusso a 2 step :

1. POST `action=search` → redirect GET con `?searchEmail=&prezzo=` → cerca l'utente per email (`cercaUserPerEmail`) e filtra le camere per prezzo (`getRoomPerPrezzo`)
2. POST `action=book` → `prenota( ... )` , transazione INSERT Reservation + INSERT Fattura + UPDATE Room=OCCUPATA

Il bottone di conferma del check-in si abilita **solo dopo** che la ricerca utente ha avuto successo, per evitare prenotazioni su email inesistenti.

8.10 CheckOutHandler

GET

POST

/checkout

Controller : `CheckOutController`

Template : `checkout.html`

Flusso a 2 step :

1. POST `action=preview` → calcola e mostra `CheckoutPreview` (giorni, totale).
2. POST `action=confirm` → `eseguiCheckout()` , transazione UPDATE Reservation + UPDATE Room + upsert Fattura.

Il totale e i giorni sono **sempre ricalcolati lato Controller**, mai presi dal form: lo stesso comportamento difensivo della CheckOutView Swing.

8.11 FatturaHandler

GET

POST

/fattura?id=N

Controller : `FatturaController`

Template : `fattura.html`

Operazioni :

- GET `?id=N` = visualizza il dettaglio della fattura N (DTO `FatturaDettaglio` , join con Reservation + User + Room)
- POST `action=paga` + `metodoId` = `paga( ... )` (INSERT Pagamento + UPDATE stato)
- POST `action=annulla` = `annulla( ... )` (marca la fattura come NON_PAGATA)

I bottoni "Paga" e "Annulla" sono visibili solo se `FatturaController.isPagabile()` ritorna `true` (stato `IN_ATTESA` oppure `NON_PAGATA`).

8.12 GestioneFattureHandler

GET

POST

/gestione-fatture

Controller : `FatturaController`

Template : `gestione_fatture.html`

Operazioni : lista filtrata per ruolo (ruolo U vede solo le sue fatture); per ogni riga pagabile c'è un bottone "Paga" che apre una modale con la select dei metodi di pagamento e un bottone "Annulla" inline.

8.13 LogsHandler

GET /logs

DAO usato : `AccessLogDao.getDettagli()` (DTO `AccessLogDettaglio`)

Template : `logs.html`

Tabella di audit di tutti gli accessi: utente, ruolo, IP, login_time, logout_time. Se `logout_time` è null mostra un badge "sessione aperta".

Accesso : solo AS e AC.

8.14 LogoutHandler

GET /logout

Controller : `LoginController.registraLogout()`

Chiama `registraLogout()` che fa UPDATE su `AccessLog.logout_time`, invalida la `sessions.token` e pulisce `SessionContext`. Poi cancella il cookie web e redirige a `/login`.

8.15 LoadingHandler e Loading1Handler

GET / e /loading1

Template : `loading.html` (welcome con bottone "Entra"), `loading1.html` (barra di progresso CSS pura + `<meta http-equiv="refresh">` a 3 secondi verso `/login`).

Sono l'equivalente dei due splash JFrame della v3 Swing. Non hanno logica server-side.

9. Template e placeholder

Tutti i template HTML vivono sotto `src/it/collegio/web/templates/` e sono letti dal classpath via `HtmlRenderer.loadTemplate()`.

9.1 Convenzione layout / contenuto

Il file `layout.html` contiene la struttura comune (DOCTYPE, navbar, footer) con due placeholder principali:

- `{{title}}` — il titolo della pagina (visibile nel tab del browser).
- `{{content}}` — lo slot in cui viene iniettato il template specifico della pagina.
- `{{user}}` — nome dell'utente loggato, visibile nella navbar.

Le pagine senza navbar (login, registration, password, loading, loading1) **non** usano `renderInLayout` ma `render` diretto, perché sono pagine "full screen" complete di `<!doctype>`.

9.2 Placeholder per pagina

Template	Placeholder principali
layout.html	{{title}}, {{content}}, {{user}}
login.html	{{title}}, {{error}}
registration.html	{{nome}}, {{cognome}}, {{email}}, {{telefono}}, {{viaOptions}}, {{domandaOptions}}, {{alert}}
password_step1.html	{{email}}, {{alert}}
password_step2.html	{{email}}, {{domanda}}, {{alert}}
home.html	{{userName}}, {{roleType}}, {{tiles}}
rooms.html	{{rows}}, {{count}}, {{alert}}, {{formBlock}}
manage_user.html	{{rows}}, {{count}}, {{alert}}, {{searchValue}}, {{formBlock}}, {{filterNote}}
manage_tenant.html	{{rows}}, {{count}}, {{alert}}, {{formBlock}}
manage_reservation.html	{{rows}}, {{count}}, {{alert}}, {{filterNote}}
checkin.html	{{alert}}, {{roomOptions}}, {{searchEmail}}, {{prezzoFiltro}}, {{userBlock}}, {{formDisabled}}, {{availableCount}}
checkout.html	{{alert}}, {{preview}}, {{reservationId}}, {{checkOutDate}}
fattura.html	{{body}}
gestione_fatture.html	{{rows}}, {{count}}, {{alert}}, {{filterNote}}
logs.html	{{rows}}, {{count}}
loading.html	{{title}}
loading1.html	{{title}}

9.3 Composizione dei placeholder complessi

Quando un placeholder rappresenta una **lista** (es. `{{rows}}`), l'handler costruisce la stringa HTML con un `StringBuilder` e la inietta:

```
StringBuilder rows = new StringBuilder();
for (Room r : rooms) {
    rows.append("<tr>")
        .append("<td>R").append(String.format("%04d", r.getId())).append("</td>")
        // ...
        .append("</tr>");
}
values.put("rows", rows.toString());
```

Per gli HTML iniettati con `{{rows}}` o simili, **non** si fa escape: il contenuto è HTML strutturale costruito dall'handler. Il valore degli attributi e i campi DB sono però sempre passati attraverso `HtmlRenderer.escape()`.

10. CSS e stile visivo

10.1 Palette

Il CSS personalizzato in `static/style.css` definisce una palette ispirata al verde dei `JButton` della v3 Swing (`new Color(0, 204, 153)`):

Variabile CSS	Valore	Uso
<code>--pmc-primary</code>	#00cc99	verde principale
<code>--pmc-primary-dark</code>	#00a07a	hover, gradients
<code>--pmc-primary-soft</code>	#d6f7ec	sfondi alert PMC
<code>--pmc-bg</code>	#f6f8fa	sfondo pagina
<code>--pmc-ink</code>	#1f2d3d	colore testo

10.2 Componenti custom

Classe	Descrizione
<code>.bg-pmc</code>	Sfondo navbar a gradient verde scuro-chiaro.
<code>.btn-pmc</code>	Bottone primario verde PMC.
<code>.btn-outline-pmc</code>	Variante outline.
<code>.bg-pmc-soft</code> / <code>.text-pmc</code>	Sfondo chiaro e testo verde scuro.
<code>.tile</code>	Card cliccabile della home con bordo sinistro colorato.
<code>.tile-grid</code>	Grid responsive auto-fill min 260px per le tile della home.
<code>.brand-logo</code>	Box quadrato 64×64 con le iniziali "PM" e gradient.
<code>.login-bg</code>	Sfondo gradient radiale usato dalle pagine senza navbar.

Tutto il resto è Bootstrap 5 standard: griglia, card, table, modal, badge, form-control, form-select.

10.3 Stampa

La pagina `fattura.html` ha un bottone "Stampa" che chiama `window.print()`. La fattura è stata pensata per essere stampabile così com'è: i bottoni di azione sono nello stesso colore della pagina e si "perdono" su stampa B/N.

11. Sessione e sicurezza

11.1 Come funziona il login

1. L'utente invia il form a `POST /login`.
2. Il `LoginHandler` chiama `LoginController.login(email, password)`.
3. Se le credenziali sono valide, il Controller (in una sola operazione):
 - Inserisce un record in `AccessLog` via `AccessLogDao.insertLog()` e ne salva il `log_id`.
 - Genera un token UUID a 64 caratteri e crea un record in `sessions` con scadenza +8h.
 - Popola `SessionContext` con utente, ruolo, committente, `log_id`, `session_id`, token.

4. Il `LoginHandler` emette un cookie `PM_SID` con un secondo UUID (per il browser) e reindirige a `/home`.

Su ogni richiesta successiva il handler chiama `WebSession.isLoggedIn()`, che ritorna `true` se `SessionContext.userCounter != null && SessionContext.sessionToken != null`.

Limite importante : `SessionContext` usa campi `static`. Va benissimo per la JFrame (un utente alla volta), ma per un server multi-client il primo utente che si logga "vince" e gli altri ereditano la sua sessione. Per questo motivo il package web è pensato come **demo monoutente da localhost**. Per un uso multi-utente reale, andrebbe sostituito `SessionContext` con una mappa `Map<String, SessionData>` indicizzata dal cookie token.

11.2 Filtro per ruolo

Le restrizioni per ruolo applicate dagli handler sono:

Endpoint	Ruoli ammessi	Implementato in
<code>/rooms</code>	AS, AC	handler (redirect a <code>/home</code> se non ammesso)
<code>/tenants</code>	AS	handler
<code>/logs</code>	AS, AC	handler
<code>/users</code>	tutti, ma U vede solo se stesso	Controller (<code>getUsersPerRuolo</code>)
<code>/reservation</code>	tutti, ma U vede solo le proprie	Controller (<code>getDettagliPerRuolo</code>)
<code>/gestione-fatture</code>	tutti, ma U vede solo le proprie	Controller (<code>getDettagliPerRuolo</code>)

I filtri sui dati personali (ruolo U vede solo se stesso) sono **sempre lato Controller**, mai lato handler. Questa è una scelta architetturale precisa: se domani aggiungiamo un terzo front-end (es. REST API), eredita gratis lo stesso filtro.

11.3 Escape HTML

Tutti i dati testuali letti dal DB e iniettati in HTML passano attraverso `HtmlRenderer.escape()`, che neutralizza `&`, `<`, `>`, `"` e `'`. Questo evita injection di markup malizioso nel caso un nome utente, un'email o una nota contengano caratteri speciali.

11.4 Protezione password

Le password sono passate al Controller in chiaro nel body POST. **Non c'è HTTPS** (il server è in HTTP puro su localhost). Per la consegna universitaria su singola macchina questo è accettabile, ma in produzione la cosa minima sarebbe attivare TLS via reverse proxy.

12. Limiti noti

Limite	Impatto	Soluzione possibile
<code>SessionContext</code> con campi static	Solo un utente alla volta puo' essere loggato in tutto il processo Java.	Sostituire con una <code>Map<token, SessionData></code> e leggere il token dal cookie ad ogni richiesta.
Validazioni live (regex live email/telefono)	Mancano i feedback rosso/verde in tempo reale che ha la versione Swing.	Aggiungere un piccolo file JavaScript con event listener sui campi.
Bootstrap caricato da CDN	Senza connessione internet l'interfaccia perde gli stili.	Scaricare Bootstrap sotto <code>static/</code> e cambiare il path del CDN nel <code>layout.html</code> .
HTTP in chiaro	Le password viaggiano in chiaro tra browser e server.	Accettabile in localhost. In rete servirebbe un reverse proxy con TLS (nginx, Caddy).
Nessuna paginazione	Tabelle con migliaia di record diventano lente da renderizzare.	Aggiungere <code>?page=&size=</code> agli handler e <code>LIMIT/OFFSET</code> nelle query del DAO.
Mini-renderer senza if / for	Per ogni lista bisogna comporre l'HTML in Java a mano.	Accettabile a questo livello, oppure migrare a Thymeleaf o Freemarker.
Nessun protezione CSRF	Un sito esterno potrebbe far compiere POST a un utente loggato.	Generare un token CSRF nel cookie, validarlo su ogni POST.

13. Estensione

Aggiungere una nuova view HTML al package richiede 3 passi.

13.1 Step 1 — creare il template

Aggiungere un file `src/it/collegio/web/templates/mio_template.html` . Può essere un frammento (sarà iniettato in `layout.html` via `renderInLayout`) o una pagina full-screen completa.

13.2 Step 2 — creare l'handler

Una classe in `handlers/` che implementa `HttpHandler` . Lo scheletro tipico:

```

public class MioHandler implements HttpHandler {

    private final MioController controller = new MioController();

    @Override
    public void handle(HttpExchange ex) throws IOException {
        if (!WebSession.isLoggedIn()) {
            HandlerUtils.redirect(ex, "/login");
            return;
        }

        // 1. (se POST) parse il form, chiama il Controller
        String alert = "";
        if ("POST".equalsIgnoreCase(ex.getRequestMethod())) {
            Map<String, String> f = HandlerUtils.parseForm(ex);
            // ... chiamata al Controller
        }

        // 2. carica i dati da mostrare
        List<Cosa> dati = controller.getDati();

        // 3. componi le righe HTML
        StringBuilder rows = new StringBuilder();
        for (Cosa c : dati) { /* ... */ }

        // 4. renderizza
        Map<String, String> values = new HashMap<>();
        values.put("rows", rows.toString());
        values.put("alert", alert);

        Map<String, String> layout = new HashMap<>();
        layout.put("user", HtmlRenderer.escape(/* nome utente */));

        HandlerUtils.sendHtml(ex,
            HtmlRenderer.renderInLayout("mio_template.html", "Titolo", values, layout
        }
    }
}

```

13.3 Step 3 — registrare la route

In `WebServer.main()` aggiungere una riga:

```
server.createContext("/mio-endpoint", new MioHandler());
```

Niente di più. Rilanciando il server l'endpoint è subito disponibile.

13.4 Regole da rispettare

- L'handler **non** deve mai aprire connessioni JDBC dirette. Tutto passa dal Controller.
- L'handler **non** deve mai contenere validazioni di business: vanno nel Controller e/o nel DAO.
- L'handler **deve** chiamare `HtmlRenderer.escape()` su ogni dato testuale letto dal DB prima di iniettarlo nel template.
- Per ogni endpoint che modifica dati, usare `POST` (mai `GET` con effetti collaterali).
- I path delle route si scrivono in minuscolo, con kebab-case se necessario (es. `/gestione-fatture`).